

FORM 5 — AI Appendix — Case Study Attachment

Case-study attachment for the ModaCo Promotion Management API. This document records how AI tools were used while building the repository.

Related repository documents: *REQUIREMENTS.md*, *ADR.md*, *README.md*, and *PERFORMANCE.md*

1. Tool Manifest

List all AI models and tools used. Please specify versions where applicable, e.g., Gemini Advanced, ChatGPT-4o, Midjourney v6.

Only tools that were used for this assignment are listed. Prisma, PostgreSQL, Redis, Docker, Express, Vitest, and similar runtime or development dependencies are not AI tools.

Model / Tool	Primary Purpose of Use	Effectiveness (1-5) & Brief Why
Cursor	In-repository coding assistant used during implementation, tests, and documentation.	Repository inspection, scoped feature discussion, test drafting, documentation drafts, and command-based verification.
ChatGPT	External system design assistant, used outside the repository.	Requirements breakdown, comparison of alternatives, and tightening of change scope before the coding was started.

2. AI Tool Usage Approach

Detail your workflow. Do not paste full raw outputs; summarize your strategy and how you guided the AI.

Phase / Specific Task	Prompting Strategy & Context Provided	Human Refinement (How did you edit or iterate?)
Requirements Analysis & Architecture Design	I provided the original case-study brief and asked the AI to separate explicit requirements, derived engineering requirements, assumptions, and open decisions.	I reviewed the proposals against the original assignment, selected Express, PostgreSQL, Prisma, Redis, and a modular monolith, and rejected unnecessary cloud-specific infrastructure. I explicitly prioritized a locally runnable solution with free dependencies and documented unresolved business decisions instead of allowing the AI to invent requirements.
Incremental Implementation & Testing	I divided the implementation into small, reviewable changes. For each change, the AI received repository context, covered requirements, expected behavior, edge cases, tests, verification commands, and explicit out-of-scope items.	I reviewed every change before accepting it, reduced prompts that attempted to cover too much in one commit, and adjusted the implementation scope to avoid overengineering. I replaced application-only promotion conflict checks with database-backed enforcement.

Reliability, Performance & Documentation Review	I used AI to identify concurrency, retry, cache-failure, pagination, decimal-price, and serverless timeout risks. Later prompts focused on ingestion retry safety, Redis fallback, performance verification, README content, ADR trade-offs.	I verified suggestions through unit, integration, and concurrency tests, Prisma migrations, linting, type checking, builds, and reproducible performance scripts where applicable. I checked generated documentation against the actual repository and removed or corrected claims that were not supported by the implementation or test evidence.
---	--	--

3. Judgement, Challenges & Verification

Highlight your critical thinking. How did you handle the limitations of the AI?

Challenge Encountered (e.g., hallucinations)	Judgement / Verification (How did you fact-check?)	Resolution (How did you fix the issue?)
An AI-suggested architecture leaned toward cloud-specific services such as managed object storage, queues, orchestration etc. Although technically valid for production, this introduced unnecessary remote dependencies and operational complexity for a locally developed assignment.	I compared the proposal with the actual brief: the ingestion processor had to be compatible with serverless constraints, but the assignment did not require deploying cloud infrastructure. I evaluated whether the essential properties, limited memory use, stateless workers and retry safety	I redirected the design to local file storage, PostgreSQL-backed import job and chunk state, workers, Redis, and Docker Compose. The workers process one chunk and exit. Possible cloud mappings were documented in the ADR instead of being added as mandatory infrastructure.
AI-generated promotion flows initially risked relying too heavily on application-level conflict checks. A “check, then insert” flow could allow two overlapping promotions to pass concurrently.	I reviewed the transaction boundaries and considered two requests executing the conflict query before either transaction committed. I then added concurrency-focused tests and checked whether the invariant was protected independently	I kept application-level validation for clear API errors but added PostgreSQL-backed transactional and constraint enforcement for same-scope conflicts. This made the database the final authority and ensured that concurrent conflicting assignments could not both succeed.

4. Overall Reflection

Estimated Ratio: *What percentage of the final case study would you estimate is directly AI-generated vs. human-crafted/heavily edited? (e.g., 30% AI / 70% Human)*

Estimated ratio: approximately 60% AI-generated and 40% human-crafted or heavily edited.

AI was used extensively to generate initial implementation drafts, tests, documentation structures, and alternative architectural approaches. The human contribution included interpreting the assignment, selecting the final architecture and technology stack, defining business assumptions, splitting the work into reviewable changes, rejecting overengineered proposals, reviewing generated code, refining prompts, validating behavior, and deciding which trade-offs and limitations were ok for the assignment.

Key Takeaway: Did using AI for this specific case study reveal any notable blind spots in the tools, or significantly alter your original approach? Briefly explain.

The main blind spot was that AI often optimized for a theoretically comprehensive production system before fully accounting for the assignment's local-development context. Its early suggestions were technically possible but sometimes introduced cost exceeded their value for this submission.

Using AI therefore changed my workflow more than it changed my engineering ownership. I moved from broad implementation requests to small items, and verification steps.

AI was most effective as an implementation and review assistant, while architectural scope, business assumptions, trade-offs, and final acceptance required human decision.